INTRODUCCIÓ ALS ORDINADORS

INFORME PRÀCTICA 3

Grau d'Enginyeria Informàtica

Universitat de Barcelona

Alumne: Cristian Prado
Grup de Pràctiques: F
Curs: 2022/2023
Semestre de primavera

INTRODUCCIÓ

En aquesta pràctica, es trobarà una sèrie d'exercicis per tal de posar en pràctica els coneixements que es tenen de Ripes, respecte la part de codi i en el moment de seguir una bona estructura en el codi d'assemblador, així com passar un codi en un llenguatge d'alt niell al llenguatge assemblador. Es presentaran en més detall les instruccions de salt de memòria, on es diferencien els salts condicionals i incondicionals. Per altra banda, en la part Teòrica-Pràctica es profunditza en els càlculs de temps d'execució o de cada cicle, juntament amb la presentació dels diferents tipus d'adreçament.

OBJECTIUS

L'objectiu d'aquesta pràctica és aprofundir en l'ús de Ripes i el llenguatge d'assemblador, de forma que es puguin realitzar amb èxit la traducció d'un codi escrit en llenguatge d'alt nivell al codi màquina. Per altra banda, es busca profunditzar més en l'ús de les instruccions de salt, tant les de salt condicional com les incondicionals, que són de vital importància per realitzar els loops. A més a més, també es busca aprendre de forma teòrica quin pot ser el temps d'execució d'un programa segons el seu processador i també tenir control respecte els diferents modes d'adreçament.

PROBLEMA 1

Els salts ens permeten executar diferents blocs de codi i així podem implementar estructures de control de flux. Exemple if: distància entre dos números enters Per calcular la distància entre dos números enters, farem la resta i llavors el valor absolut. Exemples: o El programa següent en C calcula aquesta distància. Com es faria en RISC V?

Observem el codi en el llenguatge d'alt nivell C, que és el següent:

```
int a = 5;
int b = -6;
int resultat;
if (a >= b)
        resultat = a - b;
else
```

```
        resultat = b - a;
```

Per tant, veiem que haurem de fer servir salts condicionals, per tal de poder calcular la distància entre els dos nombres que ens venen donats. L'exercici ens proporciona el següent codi:

```
.data
a: .word 5
b: .word -6
resultat: .word 0
.text
        la a0, a
        lw a1, 0(a0)
        lw a2, 4(a0)
# condició
cert:
        # branca certa
fals:
        # branca falsa
end:
        sw a3, 8(a0)
```

Veiem que el que haurem de tenir en compte és primer de tot crear la condició, mitjançant una instrucció de salt condicional i després una vegada en la posició de memòria on s'ha produït el salt, posar en les línies de codi un salt incondicional a "end", ja que no volem que després continuï en la següent posició de memòria, en el cas que la condició sigui certa.

Per tant, veiem que el codi en llenguatge d'alt nivell seria el següent en codi màquina:

```
.data
        a: .word 5
        b: .word -6
        resultat: .word 0
```

```
.text
        la a0, a
        lw a1, 0(a0)            # Carreguem a = 0(a0) a a1
        lw a2, 4(a0)            # Carreguem b = 4(a0) a a2

#Comprovem si a1>=a2 o a1<a2
        bge a1, a2, cert        # Comprovem si a1>=a2
        j fals                  # Com no s'ha complit a1>=a2, llavors és a1<a2
cert:
        sub a3, a1, a2          # fem la resta a1-a2 i guardem en a3
        j end                   # Salt incondicional a end
fals:
        sub a3, a2, a1          # fem la resta a2-a1 i guardem en a3
        j end                   # Salt incondicional a end
end:
        sw a3, 8(a0)
```

El que fa el programa el primer que fa és inicialitzar a0 amb la instrucció la a0, a. Després es carreguen els valors de a i b en els registres a1 i a2, respectivament. Això es fa mitjançant la comanda LW a1, 0(a0) i LW a2, 4(a0). Entrant en la lògica del programa, el que s'ha fet és primer comprovar si a1 >= a2, amb la instrucció de salt bge a1, a2, cert. En el cas de complir-se a1>=a2, es fa un salt a la posició de memòria "cert". En cas de no complir-se, es fa un salt incondicional a "fals", ja que si no es compleix la condició de a1>=a2, llavors vol dir que automàticament és a1<a2, i per tant es fa el salt incondicional amb la instrucció j fals. En cas de ser cert, es fa la resta a1-a2 i es guarda el resultat en el registre a3, amb la instrucció SUB a3, a1, a2. Seguidament, per evitar fer el cas "fals", es fa un salt incondicional a la posició de memòria "end", amb la instrucció de memòria j end per evitar fer les línies de codi que hi ha a la posició de memòria "fals". De forma anàloga, si és fals que a1>=a2, llavors es fa la resta a2-a1 i es guarda el resultat en el registre a3, amb la instrucció SUB a3, a2, a1 i es fa un salt incondicional a "end" amb la instrucció j end. En aquest cas, aquesta línia es podria estalviar en aquesta part del programa, ja que després de la posició de memòria "fals", va la posició de memòria

“end”, i per tant no seria necessària. Finalment, en la posició de memòria “end” es guarda el valor que conté el registre a3 en la posició de memòria que ha estat designada a “resultat”, corresponent a la posició [a0] + 8.

PROBLEMA 2

Exemple while: Fibonacci

La successió de Fibonacci és una successió matemàtica de nombres naturals tal que cada un dels seus termes és igual a la suma dels dos anteriors.

Fn = Fn-1 + Fn-2

sempre es parteix dels valors inicials

F0 = 0, F1 = 1

i es a partir del elements F0 i F1 que es genera la resta d’elements.

Exemples:

F2 = 1

F3 = 2

F4 = 3

…

F9

= 34

F10 = 55

El programa següent calcula el terme de la série de Fibonacci indicat per la variable “comptador”

(Fcomptador), i el desa a la variable “resultat”. Com es faria en RISC V?

Observem el codi en el llenguatge d’alt nivell C, que és el següent:

```
int comptador = 10;
int resultat;
int a = 0;
int b = 1;
while (comptador > 0) {
 int t = a + b;
```

```
 a = b;
 b = t;
 comptador--;
}
resultat = a;
```

L’exercici ens proporciona el següent codi màquina de Ripes:

```
.data
comptador: .word 10
resultat: .word 0
.text
        la a0, comptador
        lw a0, 0(a0)
        addi a1, zero, 0
        addi a2, zero, 1
loop:
# condició
# cos del bucle
end:
        la a0, resultat
        sw a1, 0(a0)
```

Observem que serà indispensable utilitzar una instrucció de salt incondicional al final del loop, ja que d'aquesta forma fem un salt cap enrere, per tal de repetir les línies que hi ha a la posició de memòria de "loop". Per altra banda, necessitarem un registre auxiliar per tal de poder trobar els nombres de Fibonacci, ja que haurem d'emmagatzemar primer el resultat de a1 + a2, després donar el valor de a2 a a1 i després el valor de a3 a a2. Veiem llavors el codi màquina per realitzar Fibonacci:

```
.data
comptador: .word 10
resultat: .word 0
```

```
.text
la a0, comptador            # S'assigna a a0 l'adreça de comptador
lw a0, 0(a0)                # Carreguem el valor de 0(a0) en a0
addi a1, zero, 0            # Assignem a a1 el valor 0 (F_0)
addi a2, zero, 1            # Assignem a a2 el valor 1 (F_1)

# És un loop que funciona mentre a0 > 0
loop:
   blez a0, end             # Condició per tal de continuar el loop
   add a3,a1, a2            # Definició de Fibonacci
   addi a1, a2, 0           # Serveix per guardar valor a2 en a1
   addi a2, a3, 0           # Serveix per guardar valor a3 en a2
   addi a0, a0, -1          # Es decrementa el valor de a0 en 1
   j loop                   # Salt incondicional per tal de repetir el loop
end:
   la a0, resultat          # S'assigna a a0 l'adreça de resultat
   sw a1, 0(a0)             # Es guarda el valor de a1 en 0(a0)
```

Observem que el primer que es fa és assignar a a0 l'adreça de "comptador", amb LA a0, comptador. Seguidament, es carrega al registre a0 el valor que hi ha a la posició de memòria [a0] + 0 amb LW a0, 0(a0) i també s'inicialitzen els registres a1 i a2 amb ADDI a1, zero, 0 i ADDI a2, zero, 1, els dos primers nombres de la seqüència de Fibonacci. Seguidament ve el bucle, posició de memòria la qual anomenarem “loop". Dins del bucle, el que es farà és primer de tot establir la condició de realitzar el bucle mentre que a0 > 0. En aquest cas, fem ús de la instrucció de salt condicional BLEZ a0, end, per tal de saltar a la posició de memòria "end" en el cas que a0 <= 0. Per tant, després es fa la definició d'obtenció de nombres de Fibonacci, amb ADD a3, a1, a2, per tal de fer la suma de a1+a2 i guardar-la després en el registre a3. Seguidament, toca fer una actualització dels valors dels registres. Per tant, primer de tot assignem a a1 el valor de a2 i després el valor de a3 li assignem a a2.

En aquest cas, l'ordre de les línies de codi és important, ja que en cas contrari no funcionaria el codi.

No disposem d'una instrucció d'assignació directa, com és un "=", llavors el que fem és fer ús de la instrucció ADDI a1, a2, 0. D'aquesta forma, el que estem fent és guardar en el registre a1 el valor de a2 + 0 = a2. De forma anàloga, fem el mateix amb a2 i a3, tal que ADDI a2, a3, 0. Finalment, una vegada hem actualitzat el valor dels registres, es fa un salt incondicional cap enrere, per tal de repetir les línies de codi de la posició de memòria "loop". Això ho fem amb la instrucció de salt incondicional j loop. Per tant, una vegada s'ha complit la condició de BLEZ a0, end, es fa un salt a la posició de memòria end, on el que es fa primer és assignar l'adreça de "resultat" al registre a0 amb la instrucció LA a0, resultat i seguidament guardar el valor del registre a1 en la posició de memòria [a0] + 0 amb la instrucció SW a1, 0(a0).

PROBLEMA 3

Per fer a casa: implementa les parts que falten del programa assemblador.
Exemple while amb if: màxim comú divisor
El màxim comú divisor (mcd) de dos o més nombres enters positius és el major divisor possible de tots ells.
mcd(a,b)

Exemples:

mcd(252,105) = 21
mcd(13,31) = 1

A continuació hi ha la implementació en C usant l'algorisme d'Euclides. Dins d'un bucle "while", a cada iteració es fa una comparació en un "if". Com es faria en RISC V?

Tingues en compte el flux del programa assemblador, i el que hem explicat en els exercicis anteriors sobre com avaluem de forma diferent les condicions en llenguatge d'alt nivell i en llenguatge màquina, i que algunes estructures de flux necessiten també salts incondicionals.

Observem el codi en el llenguatge d’alt nivell C, que és el següent:

```
int a = 252;
int b = 105;
int resultat;
while (a != b) {
if(a>b)
	a = a-b;
else
	b = b-a;
}
resultat = a;
```

I el codi màquina de Ripes que ens dona l'enunciat és el següent:

```
a: .word 252
b: .word 105
resultat: .word 0
.text
	la a0, a
	lw a1, 0(a0)
	lw a2, 4(a0)
loop:
	# condició while
	# condició if
cert:
	# branca certa
fals:
	# branca falsa
end:
	sw a1, 8(a0)
```

El codi màquina corresponent a l'algorisme d'Euclides seria de la forma següent:

```
.data
        a: .word 252
        b: .word 105
        resultat: .word 0
.text
        la a0, a
        lw a1, 0(a0)
        lw a2, 4(a0)

# Bucle que realitza l'algorisme d'Euclides
loop:
        beq a1, a2, end         # condició del  while, és a dir si a1 == a2, es salta a end
        bgt a1, a2, cert        # si es compleix que a1 > a2
        j fals                  # salt incondicional si no es compleix que a1 > a2, és a dir
                                  vol dir que a1 < 2
cert:
        sub a1, a1, a2          # a1 = a1-a2
        j loop                  # Salt incondicional enrere per tornar a fer el loop
fals:
        sub a2, a2, a1          # a2 = a2-a1
        j loop                  # Salt incondicional enrere per tornar a fer el loop
end:
        sw a1, 8(a0)            # Es guarda el valor de a1 en la posició de memòria [a0] +
                                  8
```

EXERCICI 1

Quines instruccions de salt condicional hem fet servir? En quin cas salten?

Per tal de fer la comprovació de si a1>=a2 o a1<a2, hem fet ús de la instrucció de salt condicional BGE a1, a2, cert, que saltava a la posició de memòria "cert" en el cas que a1>=a2. En cas contrari, seguia a la següent línia de codi.

Què fan les instruccions de salt condicional quan la condició no es compleix?

Tal i coms s'ha dit prèviament, si la condició no es compleix, no fan res, és a dir executen la següent instrucció / línia de codi.

Per què hem utilitzat les instruccions de salt incondicional?

En el cas del codi que està a l'apartat de Problema 1, s'han fet servir dos salts incondicionals, ja que si no es complia la condició de BGE a1, a2, cert, llavors volia dir que a1<a2, i llavors fèiem el cas "fals". Per tant, ja directament es podia fer un salt incondicional a la posició de memòria "fals".

Per altra banda, una vegada el programa estava en la posició de memòria "cert" o "fals", després de fer la resta, hem fet un salt incondicional a "end", amb la instrucció j end. Això era d'especial importància (degut a l'estructura del codi) en la posició de memòria "cert", ja que si no es feia aquest salt, hagués fet també les línies de codi de la posició de memòria "fals". Per tant, d'aquesta forma, amb el salt incondicional no es fan aquestes línies i directament es passa a la posició de memòria "end". També s'ha posat la línia de codi j end en la posició de memòria "fals", però després d'aquest salt, ja anava "end". Per tant no era tan necessària aquesta línia en "fals".

EXERCICI 2

Quin tipus d'estructura de control de flux indica normalment un salt cap enrere?

Un salt cap enrere és normalment indicatiu de l'estructura de control de flux anomenada bucle, el que es podria traduir en llenguatges d'alt nivell com un bucle for o while.

Omple la següent taula executant el programa.

Els valors de la taula són donats amb l'ús del processador Single Cycle Processor(Esquerra) i Pipeline de 5 etapes (dreta). Cal tenir en compte que F0 i F1 ja estan en el codi, per tant per obtenir aquests serien 0 cicles (òbviament). En la taula s'especifica el nombre de cicles i instruccions si el que volguéssim és trobar aquests nombres.

Valor inicial	Valor a "resultat"	# cicles (SC/Pipe 5)	# instruccions
F0	0	9/15	9
F1	1	15/23	15
F2	1	21/31	21
F3	2	27/39	27
F4	3	33/47	33
F5	5	39/55	39
F6	8	45/63	45
F25	75025	159/ 215	159
F46	1836311903	285/ 383	285
F47	-1323752223	291/ 391	291

Què passa amb el resultat F47?

Amb el resultat de F47 s'ha produït un overflow, i per tant en aquest cas, degut a la suma de dos positius en aquest cas, en no poder representar-se un valor més gran que el resultat d'aquesta suma, llavors el resultat ens dona un nombre negatiu.

EXERCICI 3

Descriu el programa que has implementat. Quins salts has usat? Quins són condicionals i quins són incondicionals, i per què?

El primer que es fa en el programa és assignar l'adreça de "a" en el registre a0, amb la instrucció LA a0, a. Seguidament, es carrega al registre a1 el valor que hi ha a la posició de memòria [a0] + 0 amb LW a1, 0(a0) i al registre a2 el valor que hi ha a la posició de memòria [a0] + 4 amb LW a2, 4(a0) . Seguidament ve el bucle, posició de memòria la qual anomenarem “loop". Dins del bucle, el que es farà és primer de tot establir la condició de realitzar el bucle mentre que a1 != a2. En aquest cas, fem ús de la instrucció de salt condicional BEQ a1, a2, end, per tal de saltar a la posició de memòria "end" en el cas que a1 == a2. Si no es compleix això, es comprova si a1 > a2 i es salta a la posició de memòria "cert" si es compleix, mitjançant la instrucció de salt condicional BGT a1, a2, cert. Si no es compleix això, es fa un salt incondicional a "fals", ja que voldrà dir que a1 < a2. Si es salta a "cert", llavors es fa a1 = a1-a2, és a dir es resta a1-a2 i s'assigna el valor al registre a1, mitjançant SUB a1, a1, a2. En cas que es salti a "fals", llavors es fa a2 = a2-a1, és a dir es resta a2-a1 i s'assigna el valor al registre a2, mitjançant SUB a2, a2, a1. En tots dos casos, després es farà un salt incondicional a la posició de memòria de "loop", ja que volem que es repeteixi el bucle fins que es compleixi la condició, mitjançant la instrucció j loop. Una vegada que es compleixi la condició del bucle, es salta a la posició de memòria "end", on es guarda el valor de a1 en la posició de memòria [a0] + 8, amb la instrucció SW a1, 8(a0).

Observem que s'ha hagut de fer ús de salts condicionals i incondicionals. Per tal d'anar a les posicions de memòria "cert" o "fals", s'han fet ús de salts condicionals,

concretament amb les instruccions: BEQ a1, a2, end, que era la condició del bucle, és a dir si a1 == a2 que faci el salt a "end"; BGT a1, a2, cert, que s'ha fet servir per saltar a cert, és a dir, es saltava a cert en el cas que a1>a2.

Per altra banda , com que ja s'havia comprovat prèviament si a1 == a2, a1>a2, no cal comprovar que a1<a2, ja que si no es compleixen les dues condicions anteriors, llavors s'ha de saltar a la posició de memòria "fals", i això s'ha fet amb un salt incondicional amb la instrucció j fals. Una vegada s'havien fet les comprovacions anteriors, en el cas d'anar a cert/fals, llavors per tal de seguir amb l'algorisme després de fer la resta, calia que, dins d'aquestes posicions de memòria de cert/fals, fer un salt incondicional per tal de tornar a l'inici del bucle, és a dir a la posició de memòria "loop". D'aquesta forma, funcionava el bucle del programa i una vegada s'arribava a a1 == a2, es feia el salt condicional a la posició de memòria "end".

TEÒRIC-PRÀCTIC

Exercici 1

Executa un codi similar al presentat en la taula anterior.

El codi que hem utilitzat és el següent:

```
.data
        a: .word 6
        b: .word 7
        resultat: .word 0

.text
        la a0, a
        lw a1, 0(a0)
        lw a2, 4(a0)
        add a3, a1, a2
        sub a4, a1, a2
        mul a5, a3, a4
```

sw a5, 8(a0)

Verifica l'espai que ocupa en memòria el programa .

Segons el codi que hem utilitzat, el programa Ripes ens marca que el que ocupa el nostre programa, és a dir el .text del codi, és de 32 bytes.

Executa el programa en el simulador i compara el temps d'execució per al processador single stage i el pipeline de 5 etapes. Considereu que el clock per el primer processador i el del pipeline el podem conèixer a partir de la següent diapositiva.

Pel càlcul dels temps d'execució segons els diferents processadors, tenim diferents fórmules.

Segons ens indica la diapositiva, veiem que en el cas de fer ús del Single Stage, el nombre de cicles és 8. Per tant, el càlcul del temps d'execució queda reduït a:

Temps execució = nº cicles * Temps per cicle = 8 * 925 ps = 7.4 ns

En el cas de fer ús del pipeline de 5 etapes, cal ser una mica més específic en el càlcul, ja que el temps per cicle vindrà de la següent fórmula:

$$T_c = max \left(\begin{array}{c} t_{pcq} + t_{\text{mem}} + t_{\text{setup}} \\ 2(t_{RF\text{read}} + t_{\text{mux}} + t_{\text{eq}} + t_{\text{AND}} + T_{\text{mux}} + t_{\text{setup}}) \\ t_{pcq} + t_{\text{mux}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{setup}} \\ t_{pcq} + t_{\text{memwrite}} + t_{\text{setup}} \\ 2(t_{pcq} + t_{\text{mux}} + t_{RF\text{write}}) \end{array} \right\} \begin{array}{c} \text{Fetch} \\ \text{Decode} \\ \text{Execute} \\ \text{Memory} \\ \text{Writeback} \end{array} \Bigg)$$

Després de fer els càlculs, veiem que en el cas de Fetch, tenim una durada de 300 ps; en el cas de Decode és de 550 ps; en Execute és de 300 ps; en Memory és de 270 ps i en Writeback és de 310 ps. Per tant, agafarem el temps de Decode, que és el més gran. Per tant, tenim que el temps d'execució utilitzant el pipeline és:

Temps execució = nº cicles * Temps per cicle = 13* 550 ps = 7.15 ns

Veiem, per tant, que en aquest cas és més ràpid fer ús del processador de Pipeline de 5 etapes. Com hem vist en teoria, s'observa que no pel fet de fer més cicles implica que es trigui més temps en l'execució.

Exercici 2

Feu un programa senzill en ASM que incorpori tots aquests tipus d'adreçament. Identifiqueu-lo i expliqueu-lo.

El programa que s'ha dissenyat per utilitzar aquests modes d'adreçament és el següent:

```
.data
        a: .word 7
        b: .word 3
.text
        la a0, a            # Adreçament implícit
        lw a1, 0(a0)        # Adreçament indexat
        lw a2, 4(a0)        # Adreçament indexat
loop:
        beqz a1, end        # Adreçament relatiu
        addi a1, a1, -1     # Adreçament immediat
        addi a2, a2, 2      # Adreçament immediat
        j loop              # Adreçament directe
end:
        sw a1, 0(a0)        # Adreçament indexat
```

El programa és bastant senzill, fa un bucle amb el valor de registre a1 com a comptador i per cada iteració es suma 2 al registre a2.

Trobem el cas d'adreçament implícit en la instrucció LA, a0, a , ja que veiem que la instrucció AUIPC es troba implícita en la de Load Adress.

En el cas de l'adreçament indexat, trobem diversos, però com a exemple veiem la instrucció LW a1, 0(a0). En aquest cas, la nostra direcció de memòria és la de a0 i especifiquem l'índex on volem guardar el valor, que en aquest cas és 7.

En el cas de l'adreçament relatiu, hem fet ús de la instrucció BEQZ a1, end, que és un salt condicional i que per definició, aquest salt és cap a una direcció de memòria propera a la del salt. És a dir, es suma el offset al PC, i la suma aquesta és la direcció d'una posició de memòria propera.

Trobem l'adreçament immediat en diverses línies de codi, com per exemple en el cas de ADDI a1, a1, -1, on l'operand està especificat en el camp de la posició de memòria, i en aquest cas treballem amb l'immediat -1.

En el cas d'adreçament directe, tenim el salt incondicional j loop, on el camp d'adreça especifica la posició de l'operand.

CONCLUSIONS

Com a conclusions d'aquesta pràctica, veiem que s'ha treballat en saber comprendre com es passa d'un codi escrit en llenguatge d'alt nivell a el mateix codi però en llenguatge assemblador. A més a més, aquests codis eren amb bucles i condicions, de forma que ha incentivat a l'ús i comprensió de les instruccions de salt, tant condicional com incondicional.

Per altra banda, s'ha après a calcular els temps d'execució d'un programa de Ripes segons si s'utilitza el processador Single Stage i el Pipeline de 5 etapes, comprovant que el fet de fer-ne més cicles no implica que el temps d'execució sigui més elevat. Finalment, s'ha après de forma introductòria els diferents tipus d'adreçament que es produeixen en un codi.

En definitiva, aquesta pràctica ha servit per millorar la programació en codi màquina i la comprensió de com funciona el codi d'un programa i com es comporta respecte la memòria.